Design Polymarket

Background Knowledge（背景知識）

What Is a Blockchain（High-Level for System Designers）

1. **Append-only ledger replicated across many nodes（附加式帳本，於多個節點間複製）**：

   每個 block 都包含一組 transactions，並且引用前一個 block 的 hash（確保 immutability，不可竄改性）。

2. **Consensus protocol（共識協議）**（例如 Polygon 上的 Proof of Stake）：

   用來決定哪一個被提出的 block 會成為下一個 canonical block（主鏈上的正式區塊）。

3. **Smart contracts（智慧合約）**：

   部署在鏈上的程式，用來儲存 state（狀態）並定義規則，所有節點都會以 deterministic（確定性）方式執行。

4. **Transactions（交易）**：

   用來觸發狀態變化（例如 token transfers、swaps、outcome tokens 的 mint / burn）；交易會被廣播、驗證，並被納入 blocks。

5. **Finality（最終確定性）**：

   當交易獲得一定數量的 confirmations 之後，實務上就不可逆轉，提供強而有力的結算保證。

- 上鏈缺點：
  - 慢
  - 成本高（gas）
  - 不適合高頻寫入
- 所以實務系統常見架構是：

buildmoat.org

- **Off-chain 高頻邏輯**
- **On-chain 最終結算 / 所有權 / 不可否認紀錄**

What Is Polymarket（概念性總覽）

1. **A prediction market platform（預測市場平台）**：

   使用者針對未來事件交易 YES / NO outcome tokens（例如：「候選人 X 會不會當選？」、「BTC 在六月前是否會高於 70k？」等）。

2. **Off-chain order book + on-chain settlement**：
   - 訂單下單（order placement）、撮合（matching）與價格發現（price discovery）在 off-chain 進行，以提升效能。
   - 實際的 swaps 與贖回（redemptions）則透過 smart contracts 在 on-chain 執行（CTF）。
3. **Conditional Tokens Framework（CTF）**：
   - Collateral（例如 USDC）會被鎖定在鏈上。
   - YES 與 NO tokens 會被 mint 出來並進行交易。
   - 市場結算後，勝出的 tokens 可兌換回 collateral；輸的 tokens 則變得一文不值。
4. **UMA oracle** 負責市場結算（resolve）：
   - 提供權威性的 outcome index（YES = 0 / NO = 1）。
   - 觸發最終的 payout（支付）邏輯。
5. **Polygon blockchain**：

   作為底層的 on-chain 環境，以低手續費與快速確認時間為優勢。

Trading Terminology（交易術語）

1. **Limit order（限價單）**：

   以特定價格或更好的價格買入或賣出；只有在市場價格達到該水準時才會成交，讓交易者能控制價格，但不保證一定成交。

2. **Market order（市價單）**：

   立即以當前市場上最佳可得價格買入或賣出；保證成交，但不保證最終成交價格。

3. **Bid（買價）**：

buildmoat.org

目前市場中買方願意支付的最高價格。

4. **Ask（賣價）**：

   目前市場中賣方願意接受的最低價格。

5. **Spread（價差）**：

   最佳 bid 與最佳 ask 之間的差距，代表市場流動性與交易成本。

6. **Order books（訂單簿）**：

   即時列出所有有效的買單（bid）與賣單（ask），通常依價格排序，是價格發現與撮合的基礎。

TRADE YES	PRICE	SHARES	TOTAL
	18¢	152.06	$160.33
	17¢	670.18	$132.96
Asks	16¢	118.92	$19.03
Last: 19¢	Spread: 3¢		
Bids	13¢	268.62	$34.92
	12¢	653.00	$113.28
	11¢	1,092.44	$233.45
	10¢	1,215.40	$354.99

buildmoat.org

功能性需求（Functional Requirements）

1. 所有市場皆為 **binary（二元）** 市場（YES / NO 結果）。
2. 使用者可以管理自己的 orders（訂單）：
   - 建立新訂單（create new orders）
   - 修改既有訂單（例如 price、size）
   - 取消尚未成交的訂單（cancel open orders）
3. 當使用者的 orders 在 off-chain 被撮合後，其 on-chain 的 coins 會與 outcome tokens 進行交換：
   - 透過 smart contracts 進行 collateral（例如 USDC）↔ outcome tokens（YES / NO）的交換。
   - 當市場被 resolve（結算）後，勝出的 tokens 可兌換回 coins；輸的 tokens 則在實質上被 burned（價值為 0）。

4. 使用者可以查看每個市場的 **即時價格與市場狀態（live prices and market state）**：
    - 包含 best bid / ask、last trade 等資訊。

非功能性需求（Non-Functional Requirements）

1. **Consistency for orders（訂單一致性）**：

   從使用者角度來看，order management 必須是 **strongly consistent（強一致性）** 的：

   一旦訂單被接受或取消，之後對該使用者訂單的查詢必須反映最新狀態（允許透過像 OQS 這類 read service 傳播時，存在一個小且有界的延遲）。

2. **Scalability（可擴展性）**：

   系統需能擴展以支援：

   - 約 ~20M daily active users（每日活躍使用者）
   - 每位使用者每日約 ~5 筆交易（~100M trades/day）
   - 數千個同時存在的市場（concurrent markets）

3. **Low latency（低延遲）**
   - 下單 / 取消訂單：在同一地區使用者的情境下，端到端延遲 < 200 ms（p95）。
   - 市場價格更新：從新交易 / 新訂單產生，到前端顯示給使用者，需在 sub-second（次秒級）內完成傳播。

API 設計（API design）

- 查看某個市場的即時價格（real-time pricing）

```
GET /markets/{marketId}/summary ->
{
    "marketId":"abc123",
    "bestBid":0.58,
    "bestAsk":0.61,
    "lastTrade":0.60,
```

buildnoat.org

```
    "timestamp":1732902198123
}
```

回傳指定 market 的即時市場摘要資訊，包含 best bid、best ask、最近一筆成交價（last trade）以及時間戳記。

- 建立一筆限價單（Create a limit order）

  （若為市價單 market order，則不包含 `limitPrice`）

```
POST /orders
{
    "side":"buy",// 或 "sell"
    "orderType":"limit",// 或 "market"
    "marketId":"abc123",
    "limitPrice":0.60,// limit order 必填；market order 會被
忽略
    "numShares":100
}
-> Order object，例如：
{
    "orderId":"ord_123",
    "marketId":"abc123",
    "side":"buy",
    "orderType":"limit",
    "limitPrice":0.60,
    "numShares":100,
    "filledShares":0,
    "status":"OPEN",// OPEN | PARTIALLY_FILLED | FILLED | C
ANCELED
    "createdAt":1732902198000
}
```

此 API 用於建立新訂單，回傳一個 Order object，包含訂單狀態（status）、已成交數量（filledShares）與建立時間。

- 取消訂單（Cancel an order）

```
DELETE /orders/{orderId} ->
{
```

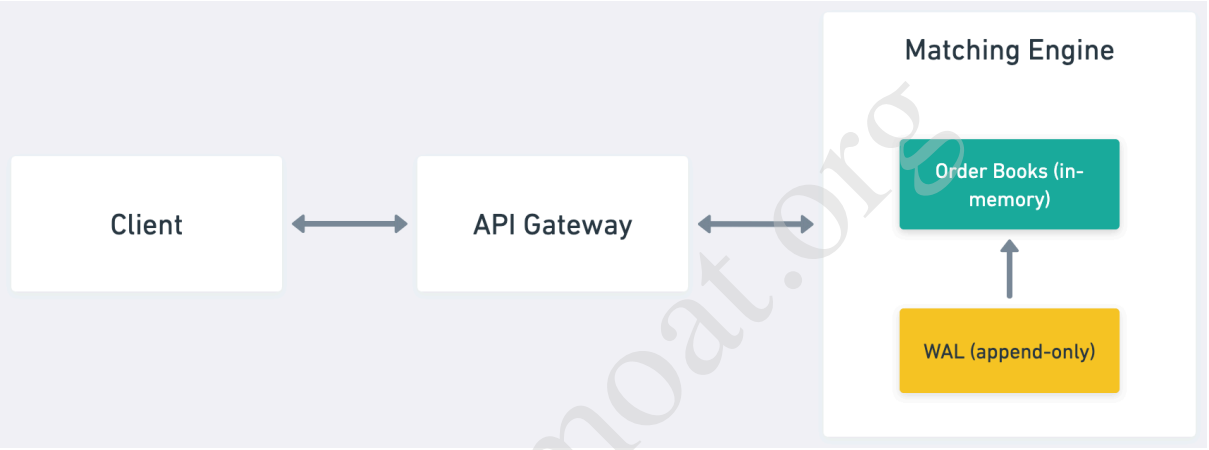
buildmoat.org

```
  "orderId":"ord_123",
  "status":"CANCELED"
}
```

用於取消指定的訂單，成功後訂單狀態會更新為 `CANCELED` 。

High-Level Design

1. Users can manage their orders（使用者可以管理訂單）



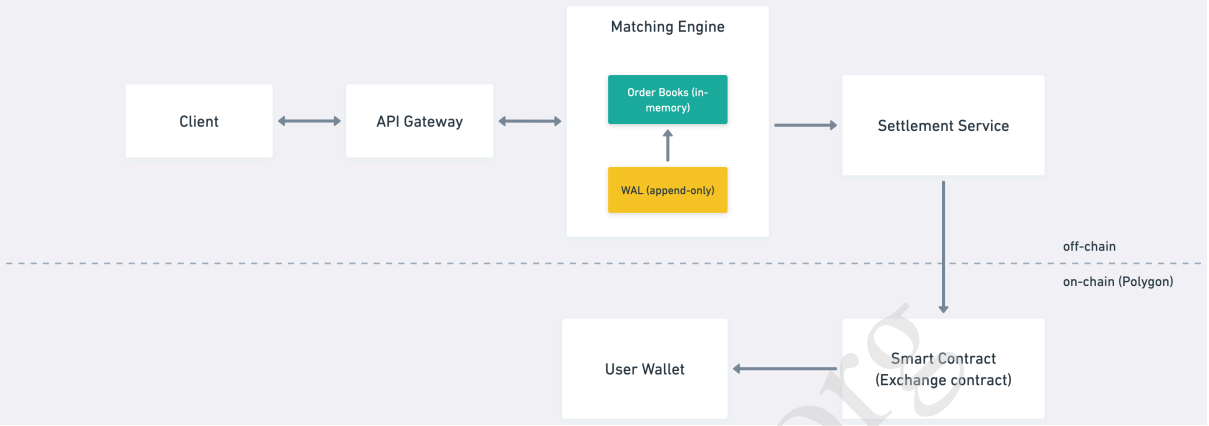
1. Client 端透過 API Gateway 將訂單提交給 Matching Engine。
2. Matching Engine 在收到使用者訂單後，會先將訂單追加寫入 write-ahead-log（WAL），接著更新不同價格層級的 in-memory order books。
   a. Order books 會放在記憶體中（而非資料庫），因為訂單撮合需要極低延遲。
   b. 必須將訂單持久化到 WAL，以便在 Matching Engine 當機時，可以從 WAL 重建 order books。
   c. Matching Engine 會定期對其 in-memory order books 進行 snapshot，並記錄對應的最後 WAL offset；在重啟時，先載入最新 snapshot，再 replay 該 offset 之後的 WAL 紀錄。
3. 當使用者嘗試修改或取消訂單時，Matching Engine 會在記憶體中定位該筆訂單，並進行更新或移除。

2. Off-chain Execution <> On-chain Settlement（鏈下執行 ↔ 鏈上結算）

buildmoat.org

當使用者的訂單成交時，需將其 on-chain coins 交換為 outcome tokens

在訂單被接受之前，Matching Engine 會先確保使用者在該市場中有足夠的 locked collateral（通常透過 smart contract，將 USDC escrow 起來並 mint outcome tokens）。



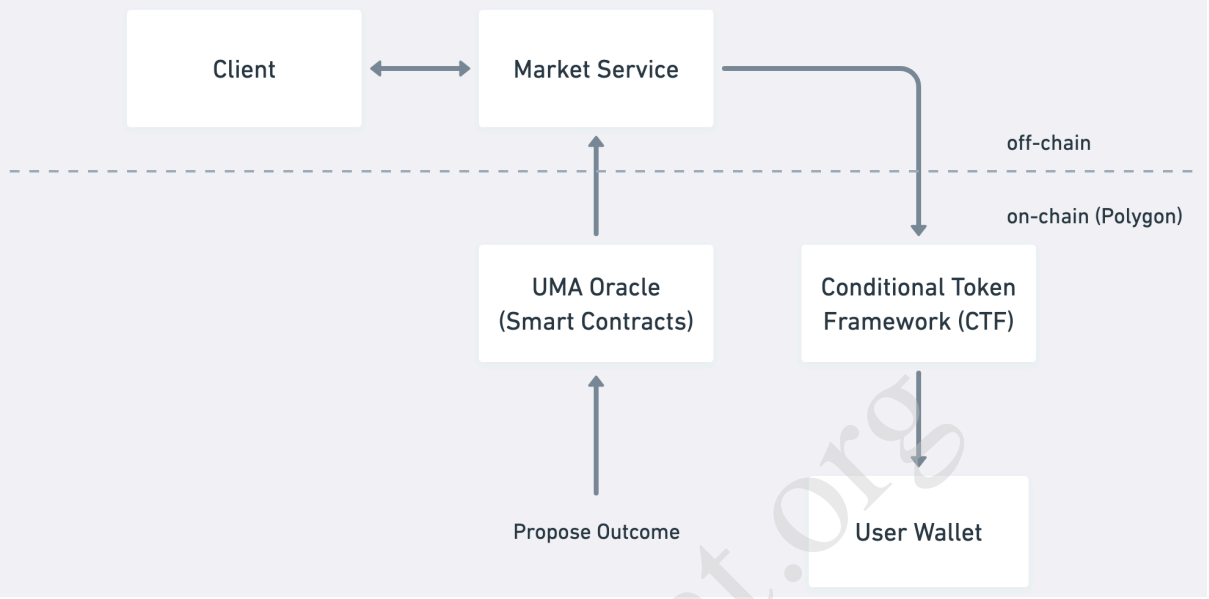
1. 當找到一組撮合（maker order + taker order）時，Matching Engine 會：
    a. 建立 order filled events，並將其追加寫入 WAL。
    b. 更新 in-memory order books。
    c. 將已撮合的訂單對提交給 Settlement Service，以準備要送往 blockchain 的交易（在 Polymarket 中是 Polygon）。
2. Settlement Service：
    a. 確認所需的 collateral / outcome-token reservation 已經存在。
    b. 透過 JSON-RPC 將交易送到 blockchain。On-chain contract（例如「Exchange contract」或「CTF Exchange contract」）負責執行 atomic swap，也就是 collateral（例如 USDC）↔ outcome-token（YES / NO）的交換。
    c. 若鏈上交易失敗，系統會自動 retry，或升級處理交由 ops。
        i. 系統在收到確認後**不會回滾（rollback）已完成的 off-chain 撮合**，因為執行流程必須是 deterministic 且 append-only。

        若進行回滾，將破壞 ordering guarantees、增加狀態重建的複雜度，並可能導致 replicas 與下游消費者之間的狀態分歧（divergence）。
3. 成交後，使用者（買單中的 taker）會在其錢包中持有 outcome tokens。

buildmoat.org

4. 為了 scalability，實際系統通常會進行 settlement batching，或事先 pre-mint positions，以避免撮合流程被鏈上吞吐量所阻塞。

當市場結果被 resolve 時，贏家兌換 token、輸家 token被銷毀



1. 某個實體（通常是 Polymarket 的 bots）會向 UMA oracle 提出市場結果。
   a. UMA（Universal Market Access）：一個被廣泛使用的協議，作為需要結果判定資料之市場的去中心化「truth machine」。
2. 在結果最終確定後，UMA oracle 會將 outcome 寫入鏈上：

```
conditionId → resolved to outcomeIndex (YES = 0 或 NO = 1)
```

1. Polymarket 監聽 oracle events，並在市場被 resolve 時通知使用者。
2. 勝出的使用者可以呼叫 Conditional Tokens Framework（CTF）來將 outcome tokens 兌換成 USDC；CTF 是另一種 smart contract，負責執行 atomic swap：
   - 若 outcome = YES → YES tokens 可兌換 1 USDC
   - 若 outcome = NO → NO tokens 可兌換 1 USDC
   - 輸的 token → 價值為 0（burned）

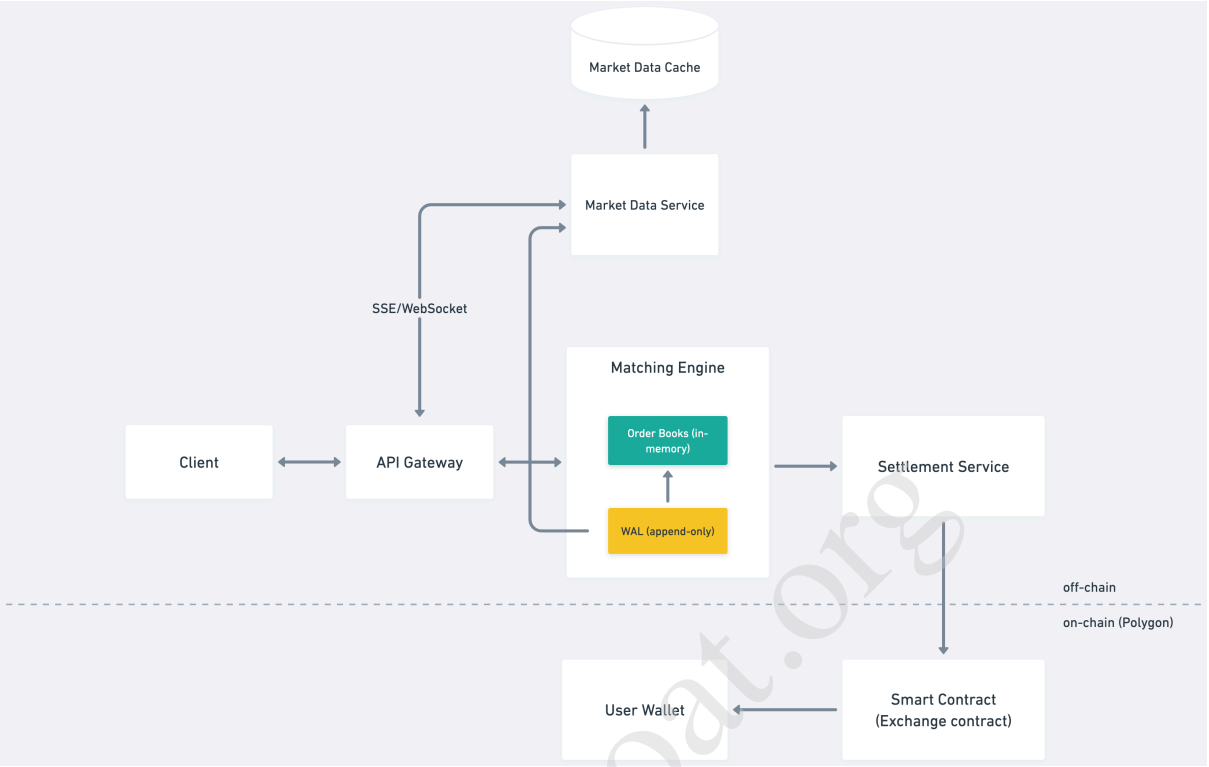
3. Users can see live prices of each market（使用者可查看每個市場的即時價格）

在許多 prediction markets 中，會將 **mid-price**

buildneat.org

`(best bid + best ask) / 2`

作為 YES 的機率顯示，同時也會呈現 last traded price 與 depth。



1. 系統中有一個獨立的 Market Data Service（MDS），用來 tail Matching Engine 的 WAL。
    a. 通常會使用 TCP streaming（或輕量級 message bus），由 Matching Engine 的 WAL publisher 將資料傳送給 MDS。
2. MDS 在接收到每一個 order event 後，會計算最新的 market data（best bid / ask、成交量等），並更新 cache。
3. 即時價格資料可拆成兩個流程：
    a. 當 frontend 第一次請求某個 market 的價格時，MDS 會從 cache 載入定價資料，並作為 initial baseline data 回傳給 client。
    b. 在透過 HTTP 傳送初始 snapshot 之後，系統會升級連線為 WebSocket（或 SSE），持續串流訂閱市場的 incremental updates。
4. 若 MDS 當機，在重啟後需要：
    a. 從 Matching Engine 取得 WAL snapshot，並從 snapshot checkpoint 之後開始重新串流 WAL。

buildmoat.org

b. 在 scale-out 架構中，MDS 會改為從 Kafka replay，而非直接連接 Matching Engine 的 WAL（此部分會在 deep dive 中說明）。

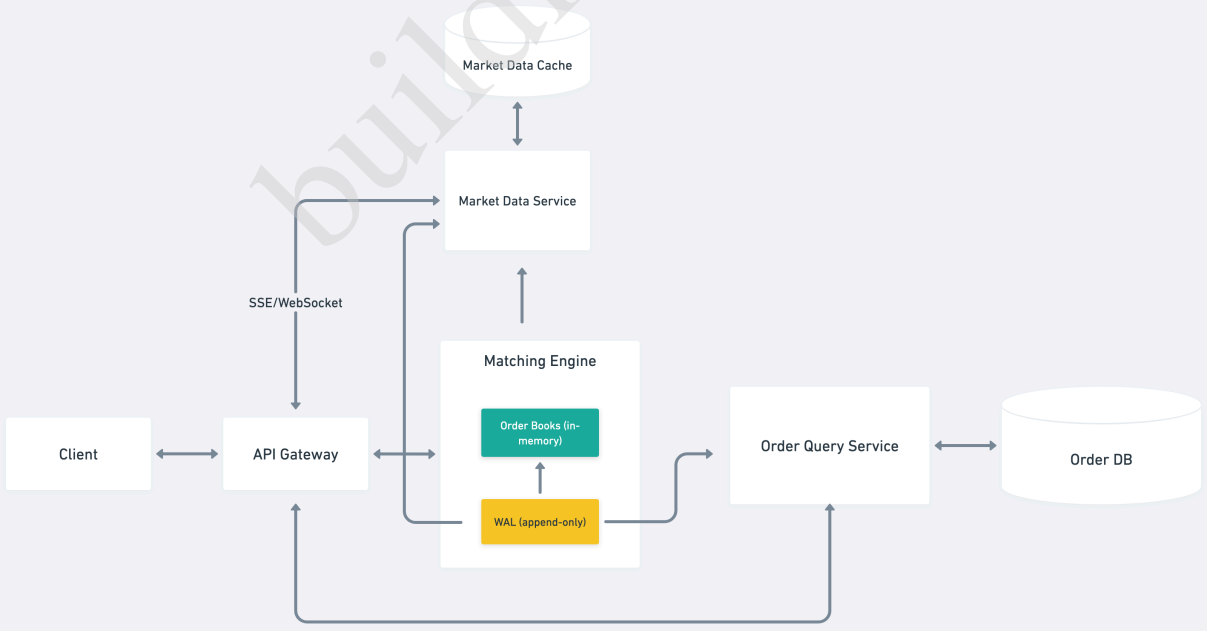
深入探討（Deep Dives）

1. How do users track their order status?（使用者如何追蹤訂單狀態？）

在 high-level design 中，我們已經說明使用者如何建立與取消訂單。然而從 UX 的角度來看，系統還必須讓使用者能夠查看並追蹤自己的訂單狀態，才能決定接下來要對訂單採取哪些行動。

為了呈現使用者的訂單資訊，其中一個做法是直接向 Matching Engine 查詢訂單資料，但這通常不是理想的選擇。Matching Engine 是為了低延遲撮合而調校的，並不適合處理下列這類成本較高的分析型查詢（analytical queries）：

- pagination（分頁）
- flexible filtering（彈性篩選）
- analytics（分析查詢）
- big joins（大型關聯查詢）



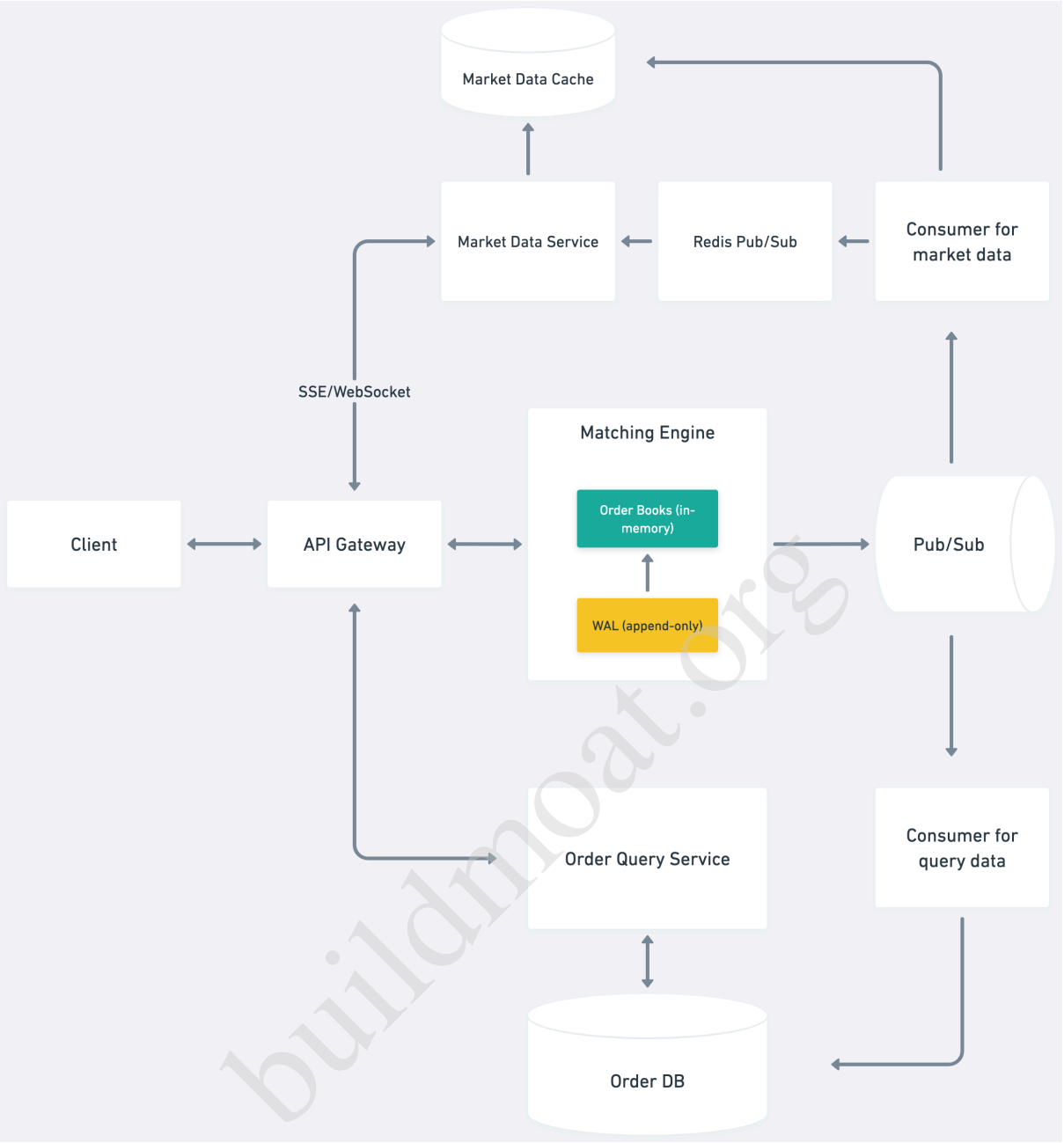
你不會希望繁重的 UI 查詢影響到撮合延遲。因此，我們會建立一個獨立的服務，稱為 **Order Query Service（OQS）**，專門用來處理訂單查詢。

buildmoat.org

- Order Query Service 會像 MDS 一樣，tail Matching Engine 的 WAL 以接收 order events。
- 在接收到訂單事件後，OQS 會將資料 materialize 到一個為查詢最佳化的資料庫中。
- 接著，client 便可以向 OQS 發出以下類型的查詢：
  - 「列出我所有尚未成交的訂單（List all my open orders）」
  - 「顯示我最近 100 筆交易（Show my last 100 trades）」
  - 「顯示 market X 的所有訂單（Show orders for market X）」

2. How does the system scale to a high number of trades per day?（系統如何擴展以支援每日大量交易？）

buildmoat.org



為了能夠讓不同服務獨立擴展（scale independently），我們首先需要將它們解耦（decouple）。在目前的設計中，MDS 與 OQS 都是直接 tail Matching Engine。若要對這三個服務進行 horizontal scaling，就必須為每個服務增加更多 instances。然而，這會導致 Matching Engine 與 MDS / OQS 之間的連線數呈指數型成長。此外，Matching Engine 還需要負責處理 MDS 與 OQS 的 downtime recovery，進一步影響其即時撮合的效能。

因此，我們在 Matching Engine 與 MDS / OQS 之間引入一層 **pub/sub layer**（例如 Kafka）。

- 當 Matching Engine 將 order events 持久化到 WAL，並更新 in-memory order books 之後，會將 order events 發佈到 pub/sub。

- Pub/Sub 可以讓同一個 event 被多個 consumers 同時消費，而每個 consumer 可服務不同用途（例如 market data、order queries）。
  - Pub/Sub 會依 `market_id` 進行 partition，以確保同一個市場內所有訂單的 ordering guarantee 與 consistency。
- 對於 **Market Data**：
  - Consumer 會根據每一個 order event 計算最新價格，並更新 market data cache。
  - 即時市場資料的推播使用 Redis Pub/Sub：
    - 每個 market 會對應到一個 Redis channel（例如 `market:<market_id>`）。
    - 只要某個 MDS server 至少有一位使用者訂閱該 market，就會 SUBSCRIBE 該 channel。
    - Redis 會在內部追蹤每個 market channel 被哪些 MDS servers 訂閱（`market → [MDS servers]`），並將更新推送給它們。
  - 每個 MDS server 會在記憶體中維護兩個 K-V 結構：
    - `market_id → [user_id]`：用來管理每個 market 的使用者訂閱清單。
    - `user_id → [market_id]`：當使用者離線時，可快速將其從已訂閱的 markets 中移除。
- 對於 **Order Queries**：
  - Consumer 會將資料 materialize 到 Order DB（例如根據 order events 更新訂單狀態）。
  - Order DB 會依 `user_id` 進行 sharding，因此同一位使用者的訂單查詢只會落在單一 shard 上。
- 在將 Matching Engine 與 MDS / OQS 解耦之後，我們便可以在必要時獨立擴展 Matching Engine。本身的 Matching Engine 也會依 `market_id` 進行 sharding，以確保每個 market 只有一個單一的 sequencer 處理其訂單。

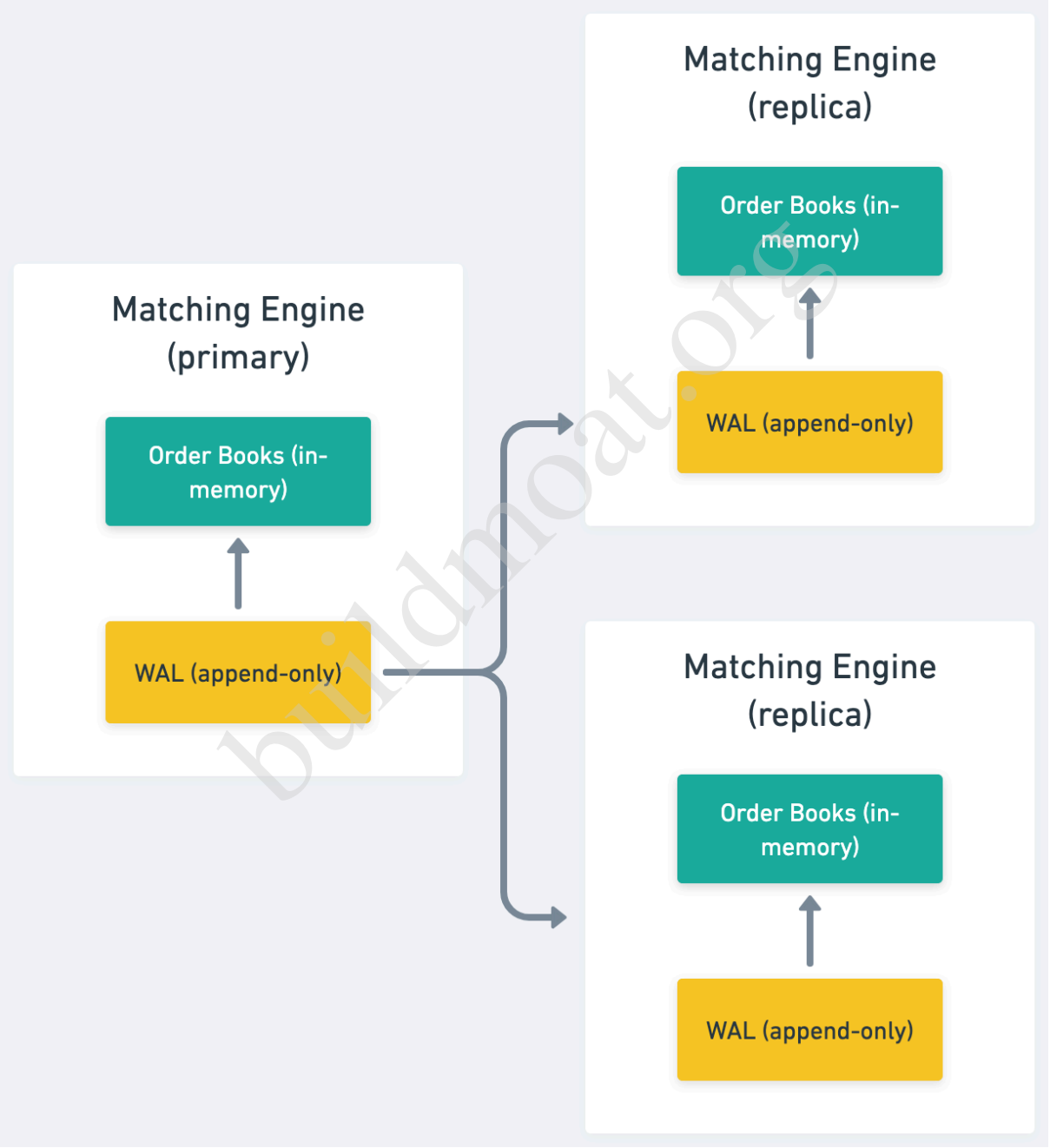
  換言之，對於任一個 `market_id`，其所屬 shard 的 WAL 就是該市場所有訂單的 authoritative event source。也因此，我們可以透過 replay WAL（或 snapshot + WAL）來重建當前 order books 的狀態，且結果必然是 deterministic（確定性的）。

3. Fault Tolerance（容錯能力）

buildmoat.org

在前一個 deep dive 中，我們已經確立：Matching Engine 承載了某一市場中 order books 的 **source of truth**，而 MDS 與 OQS 則是透過 tail 它的資料來支援不同的讀取模式（read patterns）。因此，一旦 Matching Engine 掛掉，我們就會失去權威性的市場資料。

目標：即使目前的 leader 在回應之後立刻 crash，也不能遺失任何一筆已被 acknowledged 的 order 或 fill。



為了讓系統具備足夠的穩健性（robustness），我們需要為 Matching Engine 的每個 shard 配置 replicas。整體操作流程如下：

1. Client 傳送訂單，訂單先抵達 primary instance。
2. Primary 將 log entry 追加寫入本地的 durable log。
3. Primary 透過 TCP 將 WAL entry 傳送給 replicas。
4. Replicas 將 WAL entries 套用到各自的 in-memory state machines。
5. 至少有 **1 個 replica 回傳 ACK**，表示已成功接收。這確保該事件至少存在於 **2 台機器** 上。

當 primary 發生故障時，failover 流程如下：

1. Replicas 知道最後一個已完整套用的 WAL sequence。
2. 其中一個 replica 會被提升（promote）為新的 primary。
    a. 在實務上，leader election 會由 consensus protocol（例如 Raft），或外部協調系統（例如 ZooKeeper / etcd）來負責，以避免 split-brain。
3. 新的 primary 會以新的 leader epoch 持續向 WAL 追加寫入。
4. 其他 replicas 會開始 follow 新的 leader。

所有請求在回傳「success」給使用者之前，都必須至少等待 **1 個 replica 對 WAL 寫入進行確認（ACK）**，藉此保證即使 primary 在回應後立刻 crash，也不會造成資料遺失。至於在標記請求成功前需要同步到多少個 replicas，則是 **latency 與 consistency 之間的取捨**：

replicas 越多，一致性越強，但每筆請求的延遲也會越高。

buildmeat.org